ACCESO A DATOS – 2º DAM

UP 2 – Bases de Datos Relacionales - JDBC

SESIÓN 3 - SENTENCIAS DDL/DML, INYECCIÓN SQL, MODELO POJO Y PATRÓN DAO

OBJETIVO ACADÉMICO Y PRÁCTICO (RA 2 - CRITERIOS E, F, G, H, I)
Tras consolidar la conexión a PostgreSQL, en esta sesión manipularemos los datos reales de «Mutxamel Sec-Ops S.L.». Ejecutaremos sentencias DDL y DML con Statement, prevendremos la Inyección SQL usando PreparedStatement, y gestionaremos cursores y nulos con ResultSet. Por último, organizaremos el código del proyecto creando las clases POJO y aplicando el Patrón DAO.

1. La Interfaz Statement: Ejecución de Sentencias DDL y DML

Cuando ya tenemos abierta nuestra tubería de comunicación (`Connection`) con PostgreSQL, necesitamos un vehículo para enviar órdenes SQL al servidor. El primer componente que nos ofrece la API JDBC es la interfaz `Statement` (obtenida mediante `con.createStatement()`), diseñada para ejecutar **sentencias SQL estáticas** (aquellas que son texto fijo y no reciben variables externas).

En el lenguaje SQL distinguimos diferentes sublenguajes (**DDL** para definir estructuras de tablas y **DML** para manipular o consultar datos). Dependiendo de qué tipo de orden vayamos a lanzar a `mutxamel_pg`, debemos invocar obligatoriamente uno de estos tres métodos:

Método de Statement	Sublenguaje SQL	Comportamiento y Valor de Retorno
`boolean execute(String sql)`	**DDL** (`CREATE`, `ALTER`, `DROP`) o SQL arbitrario	Ejecuta cualquier sentencia SQL. Devuelve `true` si el resultado es un conjunto de filas (`ResultSet`), o `false` si es una orden **DDL** o de modificación.
`int executeUpdate(String sql)`	**DML Modificación** (`INSERT`, `UPDATE`, `DELETE`)	Modifica el contenido de las tablas y devuelve un número entero (`int`) con la **cantidad exacta de filas afectadas** (devuelve `0` si no modificó ninguna).
`ResultSet executeQuery(String sql)`	**DML Consulta** (`SELECT`)	Ejecuta una consulta de lectura y devuelve un objeto `ResultSet` que actúa como un cursor en memoria con todas las filas recuperadas.

Arquitectura de Memoria: Liberación de Recursos y AutoCloseable
Tanto `Connection` como `Statement`, `PreparedStatement` y `ResultSet` mantienen abiertos cursores, buffers de red y planes de ejecución en el servidor PostgreSQL. Todas estas interfaces implementan `AutoCloseable`, por lo que **siempre debemos declararlas dentro de un bloque try-with-resources**. Así garantizamos que Java las cierre automáticamente en **orden inverso** a su apertura (`ResultSet` -> `Statement` -> `Connection`) en cuanto termine el bloque, incluso si salta una excepción.

2. Sentencias Preparadas (PreparedStatement) e Inyección SQL

En aplicaciones reales como la de nuestra consultora «Mutxamel Sec-Ops S.L.», las consultas incorporan variables introducidas por el usuario. Construirlas concatenando cadenas con + sobre un Statement normal está **terminantemente prohibido** por tres motivos críticos:

- **Inyección SQL (SQL Injection):** Un atacante puede alterar la estructura de la consulta para saltarse controles de acceso o destruir tablas.
- **Penalización de rendimiento:** PostgreSQL se ve obligado a reanalizar la sintaxis y recalcular el plan de ejecución en cada consulta.
- **Fragilidad:** Gestionar comillas, fechas o tipos complejos a mano genera continuos errores de sintaxis.

Anatomía de un Ataque de Inyección SQL

Imagina que programamos la búsqueda de un analista concatenando texto con un `Statement` convencional:

```
String sql = "SELECT * FROM login WHERE user_name = '" + nombreIntroducido + "'";
```

Si un usuario malintencionado escribe en el teclado la siguiente cadena tramposa:

```
Silvia'; DROP TABLE software_auditado; --
```

PostgreSQL recibirá y ejecutará dos órdenes seguidas completamente válidas:

1. `SELECT * FROM login WHERE user_name = 'Silvia';` (Cierra la comilla de forma legítima).
2. `DROP TABLE software_auditado;` (**¡Destruye por completo nuestra tabla de aplicaciones auditadas!**).

Solución de Ingeniería de Software:

Utilizar siempre **PreparedStatement** con marcadores de posición `?`. El motor de PostgreSQL precompila el árbol sintáctico de la consulta una sola vez con los huecos `?` fijos. Cuando luego enviamos el dato con `setString(1, valor)` o `setInt(1, valor)`, viaja como un literal puro y **es matemáticamente imposible que altere el comando SQL**.

3. El Cursor ResultSet y el Problema de los Valores NULL

Cuando lanzamos un `SELECT` mediante `executeQuery()`, PostgreSQL nos devuelve un objeto `ResultSet`. Puedes imaginar el `ResultSet` como una tabla en memoria con una flecha (cursor) que, al principio, apunta **justo antes de la primera fila**. Para leer los resultados usamos el bucle `while (rs.next())`: cada llamada a `next()` avanza la flecha a la siguiente fila y devuelve `true` si hay datos, o `false` cuando ya no quedan más registros.

¡Cuidado con el falso cero de los valores NULL! En una base de datos relacional, cualquier columna opcional puede contener `NULL` (ausencia de valor). Sin embargo, los tipos primitivos de Java (`int`, `double`, `boolean`) no admiten `null`. Si en PostgreSQL una columna numérica vale `NULL` y la leemos con `rs.getInt("columna")`, **JDBC nos devolverá silenciosamente un 0**, falseando los datos de nuestra auditoría.

Método de ResultSet	Comportamiento con Valores NULL y Recomendación
`rs.getString("col")`	Como `String` ya es un objeto en Java, si en PostgreSQL hay `NULL`, devuelve `null` directamente sin problema.
`rs.getInt("col")` / `rs.getDouble("col")`	Devuelve un tipo primitivo (`int` o `double`). **Peligro:** si en la base de datos hay `NULL`, retorna `0` o `0.0`, haciendo imposible distinguir un cero real de un dato vacío.
`rs.getObject("col", Integer.class)`	**Enfoque recomendado (JDBC 4.1+):** Devuelve un objeto de clase *Wrapper* (`Integer`, `Double`, `Boolean`). Si el dato en PostgreSQL es `NULL`, devuelve `null` en Java de forma limpia y exacta.

4. Capa de Modelo: Creación de las Clases POJO (`ud2.model`)

Para salvar el **desfase objeto-relacional** que estudiamos en la Sesión 2, cada tabla de nuestra base de datos `mutxamel_pg` debe tener su clase espejo en Java dentro del paquete `ud2.model`. A estas clases las llamamos **POJO** (*Plain Old Java Object*): clases limpias que no contienen sentencias SQL, sino únicamente atributos privados, constructores, *getters/setters*, `toString()` y, opcionalmente, `equals()` y `hashCode()`.

Tipo en PostgreSQL (`mutxamel_pg`)	Tipo en Java (`ud2.model`)	Extracción en JDBC (`ResultSet`)
`SERIAL` / `INTEGER` (No nulo)	`int`	`rs.getInt("id")`
`INTEGER` (Si admite NULL)	`Integer` (Clase Wrapper)	`rs.getObject("nivel_exposicion", Integer.class)`
`VARCHAR(n)` / `TEXT`	`String`	`rs.getString("user_name")`

Dentro de tu proyecto `mutxamelUP2`, haz clic derecho sobre el paquete `ud2.model` y crea las dos clases de entidad de nuestra consultora:

Paquete ud2.model -> Login.java

```
package ud2.model;

import java.sql.Timestamp;
import java.util.Objects;

public class Login {
    private int id;
    private String userName;
    private String password;
    private Timestamp createdAt;

    public Login() {}

    // Constructor para dar de alta un nuevo analista (id y created_at los asigna PostgreSQL)
    public Login(String userName, String password) {
        this.userName = userName;
        this.password = password;
    }

    // Constructor completo para reconstruir (hidratar) el objeto desde un ResultSet
    public Login(int id, String userName, String password, Timestamp createdAt) {
        this.id = id;
```

```
        this.userName = userName;
        this.password = password;
        this.createdAt = createdAt;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getUserName() { return userName; }
    public void setUserName(String userName) { this.userName = userName; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }

    public Timestamp getCreatedAt() { return createdAt; }
    public void setCreatedAt(Timestamp createdAt) { this.createdAt = createdAt; }

    @Override
    public String toString() {
        return String.format("Analista [ID=%d | Usuario=%s | Alta=%s]", id, userName,
createdAt);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Login login = (Login) o;
        return id == login.id && Objects.equals(userName, login.userName);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, userName);
    }
}
```

Paquete ud2.model -> SoftwareAuditado.java

```
package ud2.model;

public class SoftwareAuditado {
    private int codigo;
    private String nombre;
    private Integer nivelExposicion; // Usamos Integer (Wrapper) para soportar posibles NULL
de forma segura

    public SoftwareAuditado() {}

    public SoftwareAuditado(String nombre, Integer nivelExposicion) {
        this.nombre = nombre;
        this.nivelExposicion = nivelExposicion;
    }

    public SoftwareAuditado(int codigo, String nombre, Integer nivelExposicion) {
        this.codigo = codigo;
        this.nombre = nombre;
```

```
        this.nivelExposicion = nivelExposicion;
    }

    public int getCodigo() { return codigo; }
    public void setCodigo(int codigo) { this.codigo = codigo; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public Integer getNivelExposicion() { return nivelExposicion; }
    public void setNivelExposicion(Integer nivelExposicion) { this.nivelExposicion =
nivelExposicion; }

    @Override
    public String toString() {
        String riesgoTexto = (nivelExposicion != null) ? nivelExposicion + "/5" : "SIN EVALUAR
(NULL)";
        return String.format("Software [Código=%d | Nombre=%s | Riesgo=%s]", codigo, nombre,
riesgoTexto);
    }
}
```

5. Capa de Persistencia: Implementación del Patrón DAO (ud2.dao)

Para mantener una arquitectura limpia y profesional, el programa principal (`MainApp.java`) **nunca debe escribir sentencias SQL directamente**. En su lugar, aplicamos el **Patrón DAO (Data Access Object)**: creamos en el paquete `ud2.dao` una clase especialista por cada tabla, encargada de traducir objetos Java a filas de PostgreSQL y viceversa mediante las cuatro operaciones **CRUD** (*Create, Read, Update, Delete*).

5.1. Implementación de LoginDAO.java

Crea la clase `LoginDAO` dentro del paquete `ud2.dao`. Observa cómo combinamos `Statement` para lecturas generales sin filtros y `PreparedStatement` con parámetros `?` para inserciones y búsquedas seguras, apoyándonos en el diagnóstico de errores:

Paquete ud2.dao -> LoginDAO.java

```
package ud2.dao;

import ud2.model.Login;
import ud2.util.DatabaseConnection;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class LoginDAO {

    // 1. READ (Todos): Consulta estática sin parámetros usando Statement
    public List<Login> obtenerTodos() {
        List<Login> analistas = new ArrayList<>();
        String sql = "SELECT id, user_name, password, created_at FROM login ORDER BY id ASC";

        try {
```

```
            Connection con = DatabaseConnection.getConnection();
            try (Statement stmt = con.createStatement();
                 ResultSet rs = stmt.executeQuery(sql)) {

                while (rs.next()) {
                    Login a = new Login(
                        rs.getInt("id"),
                        rs.getString("user_name"),
                        rs.getString("password"),
                        rs.getTimestamp("created_at")
                    );
                    analistas.add(a);
                }
            }
        } catch (SQLException e) {
            System.err.printf("Error SQL en obtenerTodos (%s): %s%n", e.getSQLState(),
e.getMessage());
        }
        return analistas;
    }

    // 2. READ (Por Nombre): Consulta parametrizada segura frente a Inyección SQL
    public Login buscarPorUserName(String userName) {
        String sql = "SELECT id, user_name, password, created_at FROM login WHERE user_name =
?";

        try {
            Connection con = DatabaseConnection.getConnection();
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setString(1, userName); // El parámetro 1 sustituye al primer '?'

                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        return new Login(
                            rs.getInt("id"),
                            rs.getString("user_name"),
                            rs.getString("password"),
                            rs.getTimestamp("created_at")
                        );
                    }
                }
            }
        } catch (SQLException e) {
            System.err.printf("Error SQL al buscar analista (%s): %s%n", e.getSQLState(),
e.getMessage());
        }
        return null; // Si no existe, devolvemos null
    }

    // 3. CREATE: Inserción segura de un nuevo analista con PreparedStatement
    public boolean insertarAnalista(Login nuevo) {
        String sql = "INSERT INTO login (user_name, password) VALUES (?, ?)";

        try {
            Connection con = DatabaseConnection.getConnection();
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setString(1, nuevo.getUserName());
                ps.setString(2, nuevo.getPassword());
```

```
                int filasAfectadas = ps.executeUpdate();
                return filasAfectadas > 0;
            }
        } catch (SQLException e) {
            System.err.printf("Error SQL al insertar analista (%s): %s%n", e.getSQLState(),
e.getMessage());
            return false;
        }
    }
}
```

5.2. Implementación de SoftwareAuditadoDAO.java (Lectura Segura de NULL)

Ahora creamos `SoftwareAuditadoDAO` en `ud2.dao`. Fíjate en cómo empleamos `rs.getObject("nivel_exposicion", Integer.class)` para evitar el problema del falso cero si algún software aún no tiene nivel de exposición asignado:

Paquete ud2.dao -> SoftwareAuditadoDAO.java

```
package ud2.dao;

import ud2.model.SoftwareAuditado;
import ud2.util.DatabaseConnection;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class SoftwareAuditadoDAO {

    // 1. READ: Listar todo el inventario de software auditado
    public List<SoftwareAuditado> obtenerTodos() {
        List<SoftwareAuditado> lista = new ArrayList<>();
        String sql = "SELECT codigo, nombre, nivel_exposicion FROM software_auditado ORDER BY
codigo ASC";

        try {
            Connection con = DatabaseConnection.getConnection();
            try (Statement stmt = con.createStatement();
                 ResultSet rs = stmt.executeQuery(sql)) {

                while (rs.next()) {
                    SoftwareAuditado app = new SoftwareAuditado(
                        rs.getInt("codigo"),
                        rs.getString("nombre"),
                        rs.getObject("nivel_exposicion", Integer.class) // Lectura segura de
posibles NULL
                    );
                    lista.add(app);
                }
            }
        } catch (SQLException e) {
            System.err.printf("Error SQL al listar software (%s): %s%n", e.getSQLState(),
e.getMessage());
        }
        return lista;
```

```
    }

    // 2. READ (Filtrado): Buscar aplicaciones cuyo riesgo sea mayor o igual al mínimo
indicado
    public List<SoftwareAuditado> buscarPorRiesgoMinimo(int riesgoMinimo) {
        List<SoftwareAuditado> filtrada = new ArrayList<>();
        String sql = "SELECT codigo, nombre, nivel_exposicion FROM software_auditado WHERE
nivel_exposicion >= ?";

        try {
            Connection con = DatabaseConnection.getConnection();
            try (PreparedStatement ps = con.prepareStatement(sql)) {
                ps.setInt(1, riesgoMinimo);

                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        SoftwareAuditado app = new SoftwareAuditado(
                            rs.getInt("codigo"),
                            rs.getString("nombre"),
                            rs.getObject("nivel_exposicion", Integer.class)
                        );
                        filtrada.add(app);
                    }
                }
            }
        } catch (SQLException e) {
            System.err.printf("Error SQL al filtrar software (%s): %s%n", e.getSQLState(),
e.getMessage());
        }
        return filtrada;
    }
}
```

6. Puesta en Marcha en MainApp.java (ud2.app)

Actualiza tu clase `MainApp.java` dentro del paquete `ud2.app` para probar toda la arquitectura de persistencia. Observa cómo trabajamos exclusivamente con objetos `Login` y `SoftwareAuditado` (incluyendo el alta de la analista **Silvia**):

Paquete ud2.app -> MainApp.java

```
package ud2.app;

import ud2.dao.LoginDAO;
import ud2.dao.SoftwareAuditadoDAO;
import ud2.model.Login;
import ud2.model.SoftwareAuditado;

import java.util.List;

public class MainApp {
    public static void main(String[] args) {
        LoginDAO loginDAO = new LoginDAO();
        SoftwareAuditadoDAO softwareDAO = new SoftwareAuditadoDAO();
```

```
        // 1. Insertamos o verificamos a la analista Silvia en PostgreSQL
        if (loginDAO.buscarPorUserName("Silvia") == null) {
            Login silvia = new Login("Silvia", "silvia@silvia.es");
            if (loginDAO.insertarAnalista(silvia)) {
                System.out.println("Analista Silvia registrada con éxito en mutxamel_pg.");
            }
        }

        // 2. Listamos todos los analistas persistidos en la tabla 'login'
        System.out.println("\n=== PLANTILLA DE ANALISTAS (TABLA LOGIN) ===");
        List<Login> analistas = loginDAO.obtenerTodos();
        for (Login a : analistas) {
            System.out.println(a);
        }

        // 3. Listamos el inventario completo de software auditado
        System.out.println("\n=== INVENTARIO DE SOFTWARE AUDITADO ===");
        List<SoftwareAuditado> inventario = softwareDAO.obtenerTodos();
        for (SoftwareAuditado app : inventario) {
            System.out.println(app);
        }

        // 4. Consultamos aplicaciones críticas (nivel de exposición >= 4)
        System.out.println("\n=== ALERTA SEC-OPS: SOFTWARE CRÍTICO (RIESGO >= 4) ===");
        List<SoftwareAuditado> criticas = softwareDAO.buscarPorRiesgoMinimo(4);
        for (SoftwareAuditado app : criticas) {
            System.out.println(app);
        }
    }
}
```

7. Reto práctico y validación presencial

Para completar el ciclo **CRUD** en nuestra consultora **«Mutxamel Sec-Ops S.L.»**, amplía por tu cuenta la clase `SoftwareAuditadoDAO.java` programando estos tres métodos con `PreparedStatement`:

- **1. Inserción con control de nulos:**
  - public boolean insertarSoftware(SoftwareAuditado app): Inserta una nueva aplicación en software_auditado. Si app.getNivelExposicion() es null, utiliza ps.setObject(2, app.getNivelExposicion(), Types.INTEGER) para guardar el NULL en PostgreSQL sin errores.
- **2. Actualización:**
  - public boolean actualizarNivelRiesgo(int codigo, int nuevoNivel): Ejecuta una sentencia UPDATE para cambiar el nivel_exposicion de una aplicación según su codigo.
- **3. Borrado:**
  - public boolean eliminarSoftware(int codigo): Ejecuta una sentencia DELETE para borrar un registro por su clave primaria codigo.
- **4. Búsqueda por aproximación:**

- public List<SoftwareAuditado> buscarSoftwarePorNombre(String palabra): Programa un método que utilice la cláusula LIKE (o ILIKE) de SQL combinada con comodantes ("%"+palabra+"%") para encontrar y devolver todas las aplicaciones cuyo nombre contenga la palabra introducida.